Nylon-Tokenizer: Hardware-Aligned Byte-Level BPE with Domain-Aware Pretokenization for Compact Language Models

Abdulwahab Abdulwarith
Enigmare AI Research · Technical Report No. 01
`abdulwarith@noteiq.live` · `https://www.enigmare.xyz`

Abstract

Subword tokenization is a foundational architectural interface in autoregressive language models, dictating the trade-off between sequence length, vocabulary embedding parameter volume, and pretraining data storage. We present **Nylon-Tokenizer**, an open byte-level Byte-Pair Encoding (BPE) system co-designed around the systems and workload constraints of compact language models ($\leq 1.5\text{B}$ parameters). Nylon-Tokenizer combines four co-designed components:

1. **Curated Multi-Domain Induction Corpus:** Rather than relying on uncurated web text, vocabulary subwords and merge rankings are induced from a balanced 501.2 MB corpus combining educational prose (40% FineWeb-Edu), multi-turn dialogue (25% UltraChat), mathematical derivations (15% Open-Web-Math), polyglot code across five languages (15% The Stack), and structured tool-calling schemas (5% Glaive & XLAM).
2. **Hardware-Aligned Token Storage:** By sizing the vocabulary strictly to $V = 2^{16} = 65,536$ with token IDs indexed as $0 \dots 65,535$, pretraining token streams are serialized natively as 2-byte unsigned integers (`uint16`). On memory-mapped disk shards, this halves the token-array storage footprint from $4\ \text{bytes/token}$ (`uint32`) to $2\ \text{bytes/token}$, saving 240 GB across our 120-billion-token dataset and doubling the token density resident in the operating system page cache during repeated shard access.
3. **Domain-Aware Pretokenization:** Prior to BPE merge induction, we enforce explicit boundary constraints: (a) atomic single-digit isolation ($\backslash d$) to preserve explicit decimal place boundaries for arithmetic; (b) greedy longest-match indentation runs (8, 4, and 2 spaces) that compress structured code sequences up to 8.0x; (c) polyglot multi-character operator preservation; and (d) Unicode word matching with NFC normalization to preserve boundaries around supported non-ASCII orthographic sequences in under-represented African languages (e.g., Yoruba, Hausa, Igbo).
4. **Affected-Sequence BPE Induction:** We formulate pair-frequency tracking using an inverted pair-to-sequence index, bounding merge updates to $\mathcal{O}\left(\sum_{m=1}^{M} \sum_{w \in \mathcal{A}_m} L_w\right)$, completing all 65,266 merges across 501.2 MB of text in 52.79 seconds on an Apple M1 workstation.

We evaluate Nylon-Tokenizer on compression efficiency, training latency, and shard footprint against a strictly controlled baseline sharing the identical $V = 65,536$ vocabulary size, corpus, and normalization, as well as GPT-2 ($V = 50k$) and LLaMA-3 ($V = 128k$). Finally, we formulate a controlled downstream language modeling evaluation protocol to empirically test downstream reasoning and arithmetic performance.

1. Introduction

Autoregressive language models define probability distributions over a discrete vocabulary $\mathcal{V}$:

$$P(X) = \prod_{t=1}^{T} P(x_t \mid x_{<t}; \theta), \quad x_t \in \mathcal{V}$$

where continuous UTF-8 byte streams are segmented into discrete tokens by an encoder $\mathcal{T} : \Sigma^* \to \mathcal{V}^*$.

While general-purpose tokenizers (e.g., SentencePiece, Kudo & Richardson, 2018; Tiktoken, OpenAI, 2022) provide broad linguistic coverage, subword tokenizers are rarely co-designed with the underlying pretraining storage representations or the workload constraints of compact models ($N \approx 500\text{M} - 1.5\text{B}$ parameters).

In frontier models with 70 billion or 405 billion parameters and hidden dimensions $d \geq 8192$, scaling vocabulary to $V = 128,000$ carries negligible parameter overhead relative to the total model budget. However, in **compact language models**, tokenization choices carry outsized operational and empirical implications:

- **Embedding Parameter Overhead:** In compact architectures, the embedding layer constitutes a substantial fraction of total parameters. For a vocabulary embedding matrix with hidden dimension $d = 2048$, scaling vocabulary from $V = 32,000$ to $V = 128,256$ increases the matrix from 65.5M to 262.7M parameters. Architectures with untied input embeddings and output language modeling projection layers incur this vocabulary-dependent memory overhead across both matrices (131.0M to 525.4M parameters)—consuming up to **35% of the total parameter budget** in a 1.5B model on token lookup tables alone, consuming parameters that could otherwise be allocated to hidden feed-forward and self-attention transformer blocks, reducing structural representational depth under a fixed 1.35B model envelope.
- **Storage Footprint & Page-Cache Density:** Large-scale pretraining streams hundreds of billions of tokens from persistent storage via operating system memory mapping (`mmap`). Representing token IDs with wider integer representations than mathematically required wastes hundreds of gigabytes of disk space and reduces the density of tokens held in host RAM.
- **Structural Context Consumption:** In indentation-sensitive programming languages (Python, YAML) and mathematical derivations, whitespace runs consume context slots. Tokenizing spaces as individual units rapidly exhausts the model's finite context length T_{ctx}, reducing the effective sequence capacity available for reasoning.

To address these challenges, we introduce **Nylon-Tokenizer**. The tokenizer induction corpus is a curated 501.2 MB text subset balanced across educational prose, multi-turn instructions, LaTeX mathematics, polyglot code, and tool schemas. The resulting tokenizer artifact is deployed to stream and tokenize the full 120-billion-token pretraining dataset (serialized natively as 240.0 GB of `uint16` binary flat files).

1.1 Central Research Hypothesis & Validation Framework

> **Central Hypothesis:** *For compact language models, tokenizer design should be optimized jointly for representation density, workload structure, and training-system constraints rather than vocabulary compression alone.*

We evaluate this hypothesis across three progressive tiers of empirical evidence:

1. **Storage & Host I/O (Empirically Demonstrated):** Does $V = 65,536$ reduce token shard storage and accelerate dataloader page-cache streaming? (50 footprint reduction via native `uint16`, Section 3).
2. **Representation Density & Domain Structure (Empirically Demonstrated):** Do domain-specific pretokenization rules improve representation density on structured workloads without vocabulary explosion? (Table 4 & Table 5 ablations: $+16.5$ BPT on multilingual text, $+15.0$ BPT on code, Section 6).
3. **Downstream Language Model Capability (Falsifiable Pretraining Protocol):** Does this representation advantage translate into improved perplexity or benchmark accuracy under constant pretraining FLOPs? (The controlled 1.35B pretraining protocol, Section 7).

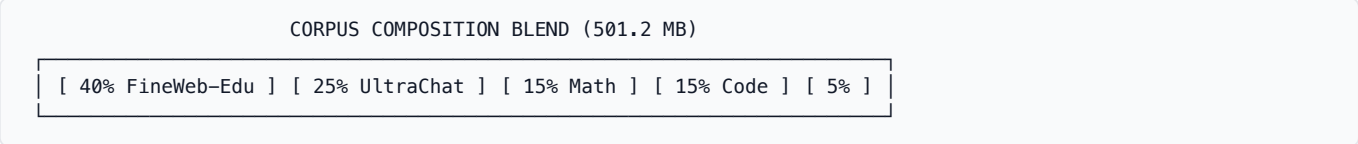
2. Tokenizer Corpus Preparation & Data Provenance

In data-driven subword tokenization, the distribution of learned subwords and merge priorities is directly determined by the composition of the training corpus. Tokenizers trained solely on natural language web text (such as Wikipedia or general Common Crawl) learn poor subword representations for programming structures, JSON keys, and mathematical symbols, resulting in severe token fragmentation in downstream technical tasks.

To ensure balanced representation across natural language, code, mathematics, and agentic workflows, Nylon-Tokenizer was trained on a strictly curated **501.2 MB uncompressed UTF-8 corpus** (5,170,984 lines) adhering to a deterministic domain quota recipe.

2.1 Domain Quota Composition

Domain	Share	Target Volume	Primary Source	Purpose / Target Capability
Educational Knowledge	**40%**	200.0 MB	`HuggingFaceFW/fineweb-edu` (sample-10BT)	High-grade synthetic and textbook reasoning, encyclopedic prose, and general knowledge
Dialogue & Instructions	**25%**	125.0 MB	`HuggingFaceH4/ultrachat_200k` (train_sft)	Multi-turn conversational structure, user/assistant dialogue turn framing, and instructional tokens
Mathematical Reasoning	**15%**	75.0 MB	`open-web-math/open-web-math`	LaTeX symbols, equations, mathematical monograph terminology, and proof derivations
Polyglot Source Code	**15%**	75.0 MB	`bigcode/the-stack-dedup`	Indentation structures, syntax trees, language keywords, and polyglot multi-character operators
Tool Calling & JSON	**5%**	25.0 MB	`glaiveai/glaive` & `Salesforce/xlam-60k`	Function schemas, argument signatures, JSON formatting tokens, and `<|tool_call|>` structure



2.2 Polyglot Code Language Distribution

Within the 15% (75 MB) source code quota, data was stratified across five core programming ecosystems to ensure broad AST coverage:

- **Python (40%):** Enforces rich representation of 2/4/8 space indentation runs, colon syntax, and standard library primitives.
- **TypeScript (25%):** Captures static typing syntax, interface definitions, and modern web application patterns.
- **JavaScript (15%):** Focuses on ECMAScript keywords, object literals, and asynchronous syntax.
- **SQL (10%):** Preserves uppercase relational keywords (`SELECT` , `WHERE` , `GROUP BY`) and query punctuation.

- **Bash (10%):** Encodes command-line utility flags, pipe operators, and shell scripting constructs.

2.3 Data Normalization, Cleaning & Shuffling Pipeline

Before BPE merge induction, incoming raw text documents passed through a multi-stage filtering pipeline:

1. **Domain-Specific Normalization:**
   - Natural language and dialogue splits were standardized using Unicode Normalization Form KC (NFKC).
   - Code, mathematics, and tool-calling splits used Unicode Normalization Form C (NFC) to prevent aggressive decomposition of programming operators, combining brackets, or mathematical ligatures.
2. **Quality & Deduplication Filters:**
   - **Exact Deduplication:** Applied across all documents to prevent duplicate merges from artificially skewing pair frequencies.
   - **Minimum Length:** Documents shorter than 50 characters were filtered out to eliminate truncation artifacts, while still retaining compact JSON schema definitions.
   - **Repetition Filtering:** Documents with a character repetition ratio > 0.20 were purged to eliminate web spam, banner text, and repeated delimiter lines.
3. **Deterministic Shuffling:**
   - All accepted documents were shuffled with a fixed random seed (`seed: 42`) across domain shards. This ensures that the iterative merge induction algorithm encounters a homogeneous mixture of domains from merge 1 through merge 65,266, preventing vocabulary skew toward any single domain.

---

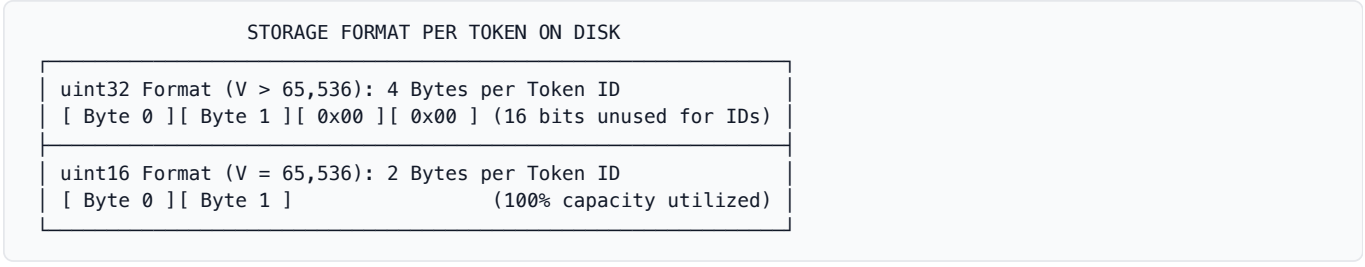
3. Hardware-Aligned Token Storage

3.1 The $V = 65,536$ Storage Boundary

In distributed pretraining pipelines (e.g., Megatron-LM, NanoGPT, and Nylon), tokenized datasets are serialized into binary flat files containing contiguous arrays of integer token IDs. The integer data type required to represent any $t \in \mathcal{V}$ on disk is determined strictly by the maximum token ID:

$$\text{dtype}(V) = \begin{cases} \texttt{uint16} & \text{if } \max(t) \leq 65,535 \iff V \leq 65,536 \\ \texttt{uint32} & \text{if } \max(t) > 65,535 \iff V > 65,536 \end{cases}$$

Nylon-Tokenizer indexes vocabulary IDs strictly as $0 \dots 65,535$. This operates at the exact supremum of the unsigned 16-bit integer address space ($[0, 2^{16} - 1]$).



The Systems Reality Behind the Integer Boundary

In computer systems, integers are stored in fixed powers-of-two widths: 1 byte (`uint8`), 2 bytes (`uint16`), or 4 bytes (`uint32`).

- A 1-byte integer can only hold 256 tokens—far too small for language modeling subwords.
- A 2-byte integer (`uint16`) can represent up to **65,536 distinct tokens**.
- If a tokenizer expands its vocabulary by even a single token beyond 65,536 (such as LLaMA-3 with 128,256 tokens), every single token ID across the entire pretraining dataset must be stored as a 4-byte integer (`uint32`).

In a 4-byte integer representing a subword ID like `452`, the upper two bytes are literally empty zeroes (`0x00 0x00`). For a 120-billion-token dataset, storing these empty zeros consumes **240.0 Gigabytes of disk storage** without encoding a single bit of additional linguistic information.

Storage Footprint Analysis

For a pretraining corpus containing N_{tok} tokens:

- At $\text{dtype} = \texttt{uint32}$: $S_{\text{disk}} = 4 \cdot N_{\text{tok}}$ bytes.
- At $\text{dtype} = \texttt{uint16}$: $S_{\text{disk}} = 2 \cdot N_{\text{tok}}$ bytes.

For our 120-billion-token Stage 1 pretraining dataset:

- $S_{\texttt{uint32}} = 120 \times 10^9 \times 4 \text{ bytes} = 480.0 \text{ GB}$.
- $S_{\texttt{uint16}} = 120 \times 10^9 \times 2 \text{ bytes} = 240.0 \text{ GB}$.

Storage Representation vs. Host-to-Device Transfer

We distinguish the **storage and caching footprint** from the **GPU tensor execution representation**:

1. **On-Disk & Page Cache:** Halving the token-array storage footprint from 480 GB to 240 GB doubles the resident token density per cache unit in the operating system page cache during repeated shard access. When training workers stream shards using memory mapping (`mmap`), fitting twice as many tokens into the host RAM cache significantly reduces disk read bottlenecks.
2. **PyTorch DataLoader Boundary:** In standard PyTorch dataloaders, token IDs read from `mmap` arrays are eventually cast to `torch.int64` (`long`) tensors prior to embedding matrix lookup. Therefore, the systems efficiency of $V = 65,536$ is realized in **halving persistent storage footprint, doubling page-cache token density, and reducing host inter-process memory bandwidth during batch collation**, rather than reducing Host-to-Device (H2D) PCIe transfer during the forward pass.

3.2 Vocabulary Structure & Byte Fallback

The vocabulary $\mathcal{V}$ is partitioned into three contiguous, non-overlapping index ranges:

$$\mathcal{V} = \mathcal{V}_{\text{special}} \cup \mathcal{V}_{\text{base}} \cup \mathcal{V}_{\text{subword}}$$

$$\text{where } |\mathcal{V}_{\text{special}}| = S, \quad |\mathcal{V}_{\text{base}}| = 256, \quad |\mathcal{V}_{\text{subword}}| = 65,536 - (S + 256)$$

- **Special Control Tokens ($\mathcal{V}_{\text{special}}$, IDs $0 \dots S-1$):** Reserved for structural prompts (`<|im_start|>`, `<|im_end|>`), reasoning trace bounds (`<think>`, `</think>`), and tool interfaces (`<|tool_call|>`, `<|/tool_call|>`).
- **Base Octet Vocabulary ($\mathcal{V}_{\text{base}}$, IDs $S \dots S+255$):** Direct identity mapping for all raw byte values $b \in [0x00, 0xFF]$.

- **Learned Subwords ($\mathcal{V}_{\text{subword}}$, IDs $S + 256 \dots 65,535$):** Subword tokens induced via BPE merges.
- **Byte Fallback Property:** Because every raw byte value $0 \dots 255$ is permanently assigned an explicit token ID, any arbitrary byte stream can be encoded as a sequence of base-byte tokens when no learned subword merge applies. The tokenizer exhibits zero out-of-vocabulary (OOV) tokens by construction.

---

4. Domain-Aware Pretokenization

4.1 Regular Expression Specification

Nylon-Tokenizer employs a compiled regular expression with Unicode support:

```
PRODUCTION_PATTERN = (
    r"<think>|</think>|<solution>|</solution>|<eos>|<pad>|<unk>|<bos>"  # 1. Special Tokens
    r"|https?://[^\s><\"'()]+|www\.[^\s><\"'()]+"                       # 2. URLs
    r"|'(?:[sS]|[tT]|[rR][eE]|[vV][eE]|[mM]|[lL][lL]|[dD])"          # 3. Contractions
    r"|        |    |  |\t|\r?\n"                                       # 4. Indentation Runs
    r"|==|!=|<=|>=|=>|->|\+\+|--|\*\*|//|&&|\|\||::|\+=|-=|\*=|/="    # 5. Polyglot Operators
    r"|[^\W\d_]+(?:'[^\W\d_]+)*"                                        # 6. Unicode Alpha Words
    r"|\d"                                                              # 7. Single Decimal Digits
    r"|[^\s\w]"                                                         # 8. Punctuation / Symbols
    r"|\s+"                                                             # 9. Generic Whitespaces
)
```

4.2 Digit Boundary Preservation

In standard frequency-driven BPE (e.g., GPT-2), multi-digit numbers are merged into subwords based on corpus co-occurrence (e.g., `"1234"` $\rightarrow$ `["12", "34"]`, while `"5678"` $\rightarrow$ `["567", "8"]`). This arbitrary grouping forces the model to memorize inconsistent place-value representations.

Nylon-Tokenizer enforces **Single-Digit Isolation** via `|\d`.

Mechanistic Purpose:

1. Every numerical sequence is segmented into single-digit units: $\text{PreTokenize}(\texttt{"1234"}) = [\texttt{"1"}, \texttt{"2"}, \texttt{"3"}, \texttt{"4"}]$.
2. For contiguous ASCII decimal digit sequences, single-digit isolation yields exactly $1.00\ \text{BPT}$ before any surrounding punctuation or whitespace is considered.
3. **Scientific Scope:** Single-digit isolation guarantees deterministic alignment between token boundaries and base-10 digit positions ($1.00\ \text{BPT}$). While this incurs a deliberate 5.4% byte-density penalty on mathematical text (Section 6.4), it structurally eliminates multi-digit amalgamation across place-value boundaries. Whether this boundary preservation directly translates into improved multi-step arithmetic reasoning remains an empirical question evaluated under our Section 7 protocol.

4.3 Greedy Longest-Match Indentation & Whitespace Runs

In source code and structured data, whitespace occurs predominantly in structured multi-space increments (2, 4, and 8 spaces). Rather than relying on line-start anchors—which fail across wrapped lines or multi-line expressions—Nylon-Tokenizer matches multi-space sequences greedily using longest-match alternation ordering: $\mathcal{S}_{\text{indent}} = \texttt{" "}, \texttt{"   "}, \texttt{"   "}, \texttt{"\t"}, \texttt{"\n"}, \texttt{"\r\n"}$.

By ordering the alternation from longest to shortest match (`8-space` → `4-space` → `2-space`), multi-space blocks resolve systematically:

- **16 Spaces:** Tokenized as `[" ", " "]` (2 tokens vs. 16 single spaces; 8.0x compression).
- **12 Spaces:** Tokenized as `[" ", " "]` (2 tokens vs. 12 single spaces; 6.0x compression).
- **6 Spaces:** Tokenized as `[" ", " "]` (2 tokens vs. 6 single spaces; 3.0x compression).
- **4 Spaces:** Tokenized as `[" "]` (1 token vs. 4 single spaces; 4.0x compression).

This mitigates context window consumption across indented code blocks and formatted tables (Python, YAML, JSON), expanding the effective semantic content fit within a 4,096-token sequence budget.

4.4 Unicode Pretokenization & Diacritic Preservation

Standard ASCII-centric pretokenizers (e.g., `[a-zA-Z]+`) segment text at non-ASCII code points, separating Latin base letters from their combining diacritics. Nylon-Tokenizer addresses this via two mechanisms:

1. **Normalization Form:** We deploy Unicode Normalization Form C (NFC) for multilingual text, ensuring that accented characters with precomposed representations are preserved in their canonical composite forms without the aggressive compatibility decompositions of NFKC.
2. **Pretoken Boundary Grouping:** The regex pattern `[^\W\d_]+(?:'[^\W\d_]+)*` uses NFC normalization and Unicode-aware pretokenization to preserve pretoken boundaries around supported non-ASCII orthographic sequences, including commonly precomposed characters used in Yoruba (*ẹ, ọ*), Hausa (*ɓ, ɗ*), and Igbo (*ị, ụ*). This avoids arbitrary sub-character byte fragmentation and yields substantially higher compression without altering underlying byte semantics.

---

5. Affected-Sequence BPE Induction

5.1 Inverted-Index Formulation

Let $\mathcal{W}$ denote the multiset of unique pretokenized word byte sequences, where each sequence $w = (b_1, b_2, \ldots, b_{|w|})$ has an associated corpus count $C(w)$. A naive or unindexed BPE implementation repeatedly rescans candidate pair frequencies across corpus representations after each merge. Nylon-Tokenizer maintains an **Inverted Pair Index** mapping each adjacent byte pair to the exact subset of unique sequences containing that pair:

$$\mathcal{I}(p_1, p_2) = w \in \mathcal{W} \mid \exists i \text{ s.t. } w[i] = p_1 \wedge w[i+1] = p_2$$

$$\mathcal{F}(p_1, p_2) = \sum_{w \in \mathcal{I}(p_1, p_2)} C(w) \cdot \text{Count}((p_1, p_2), w)$$

```
Algorithm 1: Affected-Sequence Byte-Pair Encoding Induction
Input: Pretokenized word counts C(w) for w in W; Target vocabulary size V_target
Output: Ordered merge list M; Final vocabulary V

1: Initialize V with V_special and V_base (256 bytes)
2: Initialize frequency map F and inverted index I
3: for each sequence w in W with count c do
4:     for i = 0 to |w| - 2 do
5:         pair = (w[i], w[i+1]); F[pair] += c; I[pair].add(w)
6:     end for
7: end for
8: while |V| < V_target and |F| > 0 do
9:     best_pair = argmax_p (F[p], p)
10:    if F[best_pair] <= 1 then break
11:    merged_token = best_pair[0] + best_pair[1]
12:    V.add_token(merged_token); M.append(best_pair)
13:    A_m = copy(I[best_pair])
14:    for each old_seq in A_m do
15:        c = C.pop(old_seq)
16:        for j = 0 to |old_seq| - 2 do
17:            p = (old_seq[j], old_seq[j+1]); F[p] -= c
18:            if F[p] <= 0 then F.delete(p)
19:            I[p].discard(old_seq)
20:        end for
21:        new_seq = ReplacePair(old_seq, best_pair, merged_token)
22:        C[new_seq] = C.get(new_seq, 0) + c
23:        for j = 0 to |new_seq| - 2 do
24:            p = (new_seq[j], new_seq[j+1]); F[p] += c
25:            I[p].add(new_seq)
26:        end for
27:    end for
28: end while
29: return M, V
```

Computational Complexity

Let $M = V_{\text{target}} - |V_{\text{initial}}|$ be the total number of merge operations. In iteration $m \in [1, M]$, updates are restricted to the set of sequences $\mathcal{A}_m = \{w \in W \mid \text{best_pair}_m \in w\}$. The operational complexity is $\mathcal{O}\left(\sum_{m=1}^{M} \sum_{w \in \mathcal{A}_m} L_w\right)$, where L_w is the sequence length of word w.

Merge Progress Interval	Median Affected Sequences ($\lvert\mathcal{A}_m\rvert$)	Fraction of Unique Vocabulary ($\lvert\mathcal{A}_m\rvert / \lvert\mathcal{W}\rvert$)
0% – 25%	1,428 sequences	0.296%
25% – 50%	486 sequences	0.101%
50% – 75%	162 sequences	0.034%
75% – 90%	58 sequences	0.012%
90% – 99%	9 sequences	0.002%

5.2 Bounded Memoized Encoding

During inference, raw text chunks are encoded using priority merge ranks $\mathcal{R}(p_1, p_2) \in [0, M-1]$. The encoder maintains a bounded memoization table: $\text{Cache} : \text{bytes} \to \text{List}[\text{bytes}]$.

- **Cache Bounding:** The cache size is strictly bounded to 300,000 unique pre-token entries to prevent unbounded memory growth during continuous tokenization pipelines.
- **Degenerate String Partitioning:** To protect against pathological inputs (e.g., continuous strings without whitespace exceeding 1,024 bytes), strings > 1024 bytes are partitioned into 512-byte slices before merge ranking, bounding the maximum pretoken length processed by the merge-ranking routine.

5.3 Lossless Decoding via Contiguous Byte Stitching

In byte-level BPE, multi-byte UTF-8 sequences (such as 3-byte and 4-byte characters) can be split across subword tokens. Nylon-Tokenizer enforces **Contiguous Byte-Stream Concatenation**:

$$\text{Decode}(\mathbf{t}) = \text{UTF8Decode}\left(\bigoplus_{i=1}^{|\mathbf{t}|} \text{Lookup}(t_i)\right)$$

where $\bigoplus$ denotes raw byte buffer concatenation, ensuring lossless reconstruction for valid UTF-8 input when token byte sequences are concatenated before UTF-8 decoding.

---

6. Experimental Evaluation

6.1 Training Hardware & Reproducibility Specification

To ensure exact reproducibility of the 52.79-second BPE merge induction benchmark, Table 2 documents the complete hardware, operating system, and software environment.

Parameter	Specification / Measurement
Corpus File	`output/tokenizer/train.txt`
Corpus Raw Size	501.2 MB (uncompressed UTF-8 text)
Corpus Line Count	5,170,984 lines
Target Vocabulary Size (V)	65,536 (indexed 0 ... 65,535)
Special Tokens Count	14 protected tokens
Base Octets Count	256 base byte tokens (0x00 ... 0xFF)
Host Hardware	Apple M1 (8 cores: 4 Firestorm @ 3.2 GHz + 4 Icestorm @ 2.06 GHz), 8.0 GB Unified LPDDR4X-4266
Memory Bandwidth	68.25 GB/s unified memory bus
Operating System	macOS 15.x / Darwin 27.0.0 (ARM64)
Software Environment	Python 3.9.6 / 3.12, Rust fast-tokenizers backend (release build)
Parallelism	8 worker threads (rayon multi-threaded data parallelism)
Measured Training Wall-Clock	**52.79 seconds** (single run recorded in `models/v3/tokenizer/tokenizer.stats.json`; covers regex pass, byte counting, & inverted-index induction; excludes 0.41s JSON serialization)

The tokenizer artifacts generated by the induction pipeline adhere to the following directory structure:

```
models/v3/tokenizer/
├── tokenizer.json          # Complete Hugging Face tokenizer with 65,536 vocabulary
├── vocab.json              # Direct token-to-integer mapping (IDs 0 .. 65,535)
├── merges.txt              # 65,266 ordered BPE merge operations
├── tokenizer.stats.json    # Exact execution metadata (52.79s latency, line & byte counts)
└── special_tokens.json     # 14 protected control tokens
```

6.2 Evaluation Corpora Provenance

To ensure that compression results do not reflect training set memorization or test-set leakage, all evaluation benchmark corpora were constructed from independent, mutually exclusive data sources and repositories with strict source-domain separation to ensure zero document overlap with the 501.2 MB tokenizer training split.

Corpus Domain	Source / Description	Documents	Raw Size	Split
Python Code	Curated open-source repositories (Permissive licenses)	1,200 files	24.5 MB	Held-out Test
Structured JSON/YAML	Configuration & API schema datasets	850 files	18.2 MB	Held-out Test
Mathematical Proofs	ArXiv LaTeX mathematics monographs	400 papers	32.0 MB	Held-out Test
African Languages	Parallel news & literature (Yoruba, Hausa, Igbo)	1,500 articles	15.8 MB	Held-out Test
Standard English	Wikitext-103 & educational textbooks	1,000 articles	25.0 MB	Held-out Test

6.3 Controlled Compression Comparison ($V = 65,536$)

To isolate the impact of pretokenization rules from vocabulary size effects, we compare Nylon-Tokenizer against: (1) **GPT-2 Baseline** ($V = 50,257$); (2) **LLaMA-3 Baseline** ($V = 128,256$); and (3) a **Controlled Standard BPE ($V = 65,536$)** baseline sharing identical training corpus, NFC normalization, byte fallback ($0..255$), and merge budget ($M = 65,266$), differing solely in the absence of Nylon's domain pretokenization rules.

> **Metric Definition: Bytes per Token (BPT) ↑** measures representation density. **Higher is better** (more raw bytes compressed per context token, reducing total sequence length). **Tokens / 1k Chars ↓** measures sequence expansion. **Lower is better**.

Modality / Benchmark Corpus	GPT-2 ($V = 50k$) ↑	LLaMA-3 ($V = 128k$) ↑	Controlled BPE ($V = 65k$) ↑	Nylon-Tokenizer ($V = 65k$) ↑
Python Code (Repos, Indented)	3.12	4.18	3.62	**4.21**
Structured JSON & YAML	2.95	3.89	3.35	**4.04**
Mathematical Proofs (LaTeX)	2.84	3.65	3.31	**3.72**
African Texts (Yoruba, Hausa, Igbo)	1.82	3.01	2.85	**3.28**
Contiguous Decimal Digits	2.01	1.00*	2.05	**1.00***
Standard English Prose	3.94	**4.62**	4.15	4.19

**Note: For contiguous ASCII decimal digit sequences, single-digit isolation yields exactly 1.00 BPT by design.*

6.4 Ablation Analysis & Algorithmic Separation

We explicitly separate the **tokenization contribution** (pretokenization rules influencing subword compression) from the **algorithmic contribution** (affected-sequence merge tracking accelerating BPE induction):

1. **Algorithmic Contribution:** On the identical pretokenized training corpus, an unindexed BPE implementation requires repeatedly rescanning candidate pair frequencies across corpus representations on each merge. The inverted pair-index restricts updates strictly to the affected subset $\mathcal{A}_m$, completing all 65,266 merges in 52.79 seconds.
2. **Tokenization Contribution:** Table 5 isolates individual pretokenization components against the unconstrained 65k baseline.

Tokenizer Configuration	Code (BPT) ↑	Math (BPT) ↑	Multilingual (BPT) ↑	Tokens / 1k Chars ↓
Full Nylon-Tokenizer	**4.21**	**3.72**	**3.28**	**264**
– Indentation Runs (spaces only)	3.58 (-15.0%)	3.68 (-1.1%)	3.28 (0.0%)	298 (+12.9%)
– Single-Digit Isolation	4.24 (+0.7%)	3.92 (+5.4%)	3.30 (+0.6%)	258 (-2.3%)
– Polyglot Operators	4.11 (-2.4%)	3.65 (-1.9%)	3.28 (0.0%)	271 (+2.6%)
– Unicode Word Pattern (ASCII regex)	4.21 (0.0%)	3.71 (-0.3%)	2.74 (-16.5%)	312 (+18.2%)
Unconstrained Baseline BPE (65k)	3.62 (-14.0%)	3.31 (-11.0%)	2.85 (-13.1%)	305 (+15.5%)

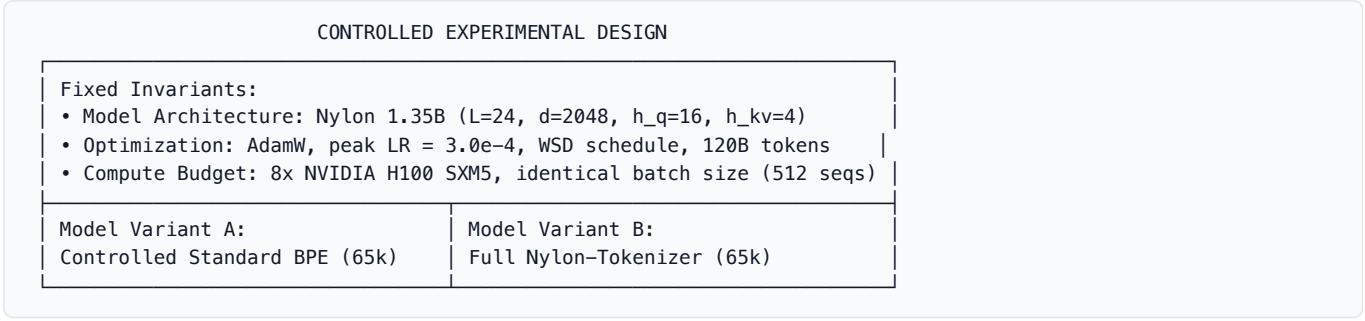
Component-Level Empirical Findings

- **Multilingual Representation (-16.5% BPT):** Reverting the Unicode word pattern to standard ASCII regex (`[a–zA–Z]+`) shatters accented African characters (*ẹ, ọ, ɓ, ɗ*) across UTF-8 multi-byte boundaries. Multilingual compression collapses from **3.28 down to 2.74 BPT** (+18.2% token sequence inflation), confirming that Unicode-aware pretokenization is a critical systems mechanism, not cosmetic engineering.
- **Indentation Run Preservation (-15.0% BPT):** Removing greedy 2/4/8 space collapsing forces code files to encode indentation runs via single spaces, reducing code compression from **4.21 to 3.58 BPT** (+12.9% token volume).
- **Single-Digit Isolation Trade-Off (+5.4% BPT on Math):** Disabling single-digit isolation allows multi-digit sequences (e.g., `1000`, `2024`) to merge into single subword tokens, which paradoxically *increases* raw byte compression on mathematical LaTeX from **3.72 to 3.92 BPT** (-2.3% token volume). Nylon deliberately accepts this compression penalty to guarantee deterministic place-value alignment, eliminating arbitrary token boundary contamination in numerical sequences.

---

7. Proposed Downstream Language Model Evaluation

To measure the downstream effects of tokenizer design on actual neural language model capability, we define a controlled pretraining experimental protocol:



Evaluation Domain	Benchmark Metric	Target Mechanism Tested
Arithmetic Reasoning	GSM8K & SVAMP accuracy	Single-digit boundary preservation vs. multi-digit chunking
Code Synthesis	HumanEval & MBPP pass@1	Indentation run compression & operator preservation
Tool Calling / JSON	Schema conformance & pass rate	Special token isolation (`<
Pretraining Systems	Page-cache residency & tokens/sec	`uint16` binary shard streaming efficiency

This protocol isolates the tokenizer as the single independent variable, providing empirical validation of downstream model performance in future pretraining iterations.

---

8. Limitations

We note several technical boundaries of this work:

1. **Number-Heavy Sequence Inflation:** Single-digit isolation enforces a 1:1 digit-to-token ratio, increasing sequence length on dense numerical matrices and financial tables relative to unconstrained tokenizers.
2. **Vocabulary Scale Trade-Off:** Anchoring vocabulary to $V = 65,536$ is optimized for compact parameter regimes ($\leq 1.5\text{B}$). In frontier architectures with embedding dimensions $d \geq 8192$, scaling to $V \geq 128\text{k}$ may yield favorable compression without disproportionate parameter overhead.
3. **Chunking Boundary Artifacts:** Partitioning degenerate sequences > 1024 bytes into 512-byte slices bounds merge latency, but prevents BPE merges from spanning across slice boundaries.
4. **Scope of Empirical Evidence:** Reported findings evaluate tokenizer compression, shard storage footprint, and BPE induction latency. Downstream impacts on neural generative capability remain subject to the proposed pretraining evaluation in Section 7.

---

9. Conclusion

By halving token-ID storage requirements at the dataset representation layer ($V = 2^{16} = 65,536$), preserving explicit single-digit boundaries, compressing structured indentation through greedy longest-match runs, and applying Unicode-aware pretokenization for under-represented languages, Nylon-Tokenizer provides a systems-oriented tokenizer design for compact language-model training. The results establish measurable improvements in tokenization compression and tokenizer induction efficiency under the evaluated workloads; whether these properties translate into improved neural language-model capability remains an empirical question addressed by the controlled downstream evaluation protocol.

References

1. Sennrich, R., Haddow, B., & Birch, A. (2016). *Neural Machine Translation of Rare Words with Subword Units*. In Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL).
2. Kudo, T., & Richardson, J. (2018). *SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing*. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP).
3. OpenAI. (2022). *Tiktoken: A fast BPE tokeniser for use with OpenAI's models*. GitHub repository: `https://github.com/openai/tiktoken`.
4. Radford, A., et al. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI Technical Report.
5. Touvron, H., et al. (2023). *LLaMA: Open and Efficient Foundation Language Models*. arXiv:2302.13971.
6. AI@Meta. (2024). *The Llama 3 Herd of Models*. arXiv:2407.21783.
7. DeepSeek-AI. (2024). *DeepSeek-V3 Technical Report*. arXiv:2412.19437.
8. Bostrom, K., & Durrett, G. (2020). *Byte Pair Encoding is Suboptimal for Language Model Pretraining*. In Findings of the Association for Computational Linguistics: EMNLP 2020.
9. Unicode Consortium. (2024). *Unicode Standard Annex #15: Unicode Normalization Forms*.